SWE3202: SOFTWARE TESTING, INTEGRATION AND QUALITY ASSUARANCE

LECTURE 3: Debugging and Testing Tools.

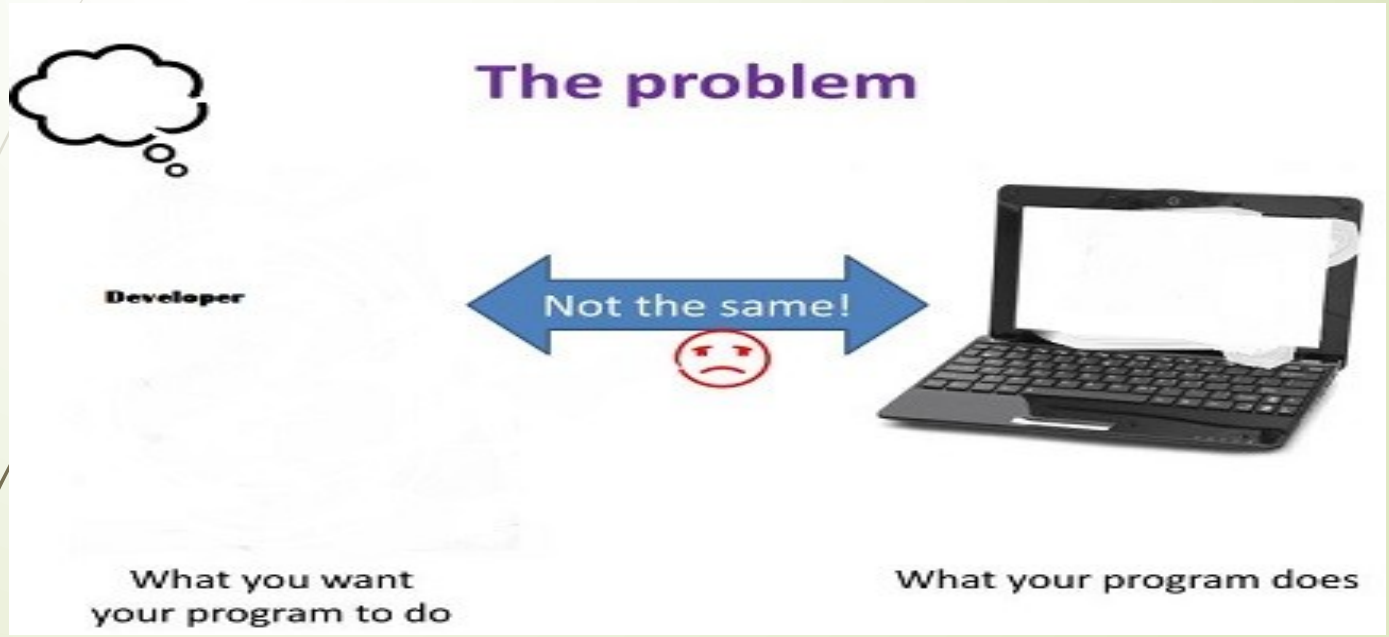
N. B Rabiu

15/6/2026

Lecture Outline

- Debugging.
- Common types of Bugs.
- Process of Debugging.
- Example of programs to debugs.
- Software Testing Tools

Introduction



Introduction

- Consequence of successful testing leads to debugging. That is, when a test case uncovers an error, debugging is the process that results in the removal of the error.
- Debugging has the following objectives:
  - *determining the exact nature, location of a suspected error and fixing it.*
- Locating the error is often 95% of the work.

Sources of Bugs

- Bad design
  Wrong/incorrect solution to problem
- Insufficient isolation
  As a result of poor encapsulation change in one area affect another.
- Typos
  Enter wrong text, choose wrong variable
- Later changes/fixes that are not complete
  Thus, changes in one might affect another

Debugging

- Debugging is *a systematic process of finding and reducing the number of bugs or defects, in a computer program* i.e. it is a process of detecting and removing the existing and potential errors (bugs) in a software code that can cause it to behave unexpectedly or even crash.
- Using information from the program tests, debuggers use their knowledge of the programming language and the intended outcome of the test to locate and repair the program error

Testing VS Debugging

Testing	Debugging
Goal: Find the *existence* of bugs	**Goal:** Find the *location & cause* of bugs
When: During development, before deployment	**When:** After a test fails (or a user reports a problem)
Output: Pass/Fail report, stack trace	**Output:** Fixed code, root cause analysis
Analogy: Medical screening (checking vitals)	**Analogy:** Diagnostic surgery (finding the blockage)

Common Types of Bugs

- Programs, when written, are expected to function properly and generate the expected output.

- However, programmers often face three types of errors — syntax errors, runtime errors, and logical errors — at different stages of software development.

Syntax Error

1. Syntax errors are deviations from the standard format specified by programming languages. These are explicitly identified by compilers and are easy to fix. Example

   - let's say the correct syntax for printing something is print('hello'), and we accidentally forget one of the parentheses while coding print ("hello",
   - a syntax error will occur, and this will stop the program from running.

- Syntax error: Are easy to find and correct because IDEs (compiler) indicate
  - where they came from and why they occur.
  - Suggest how to fix it.

Logical Error

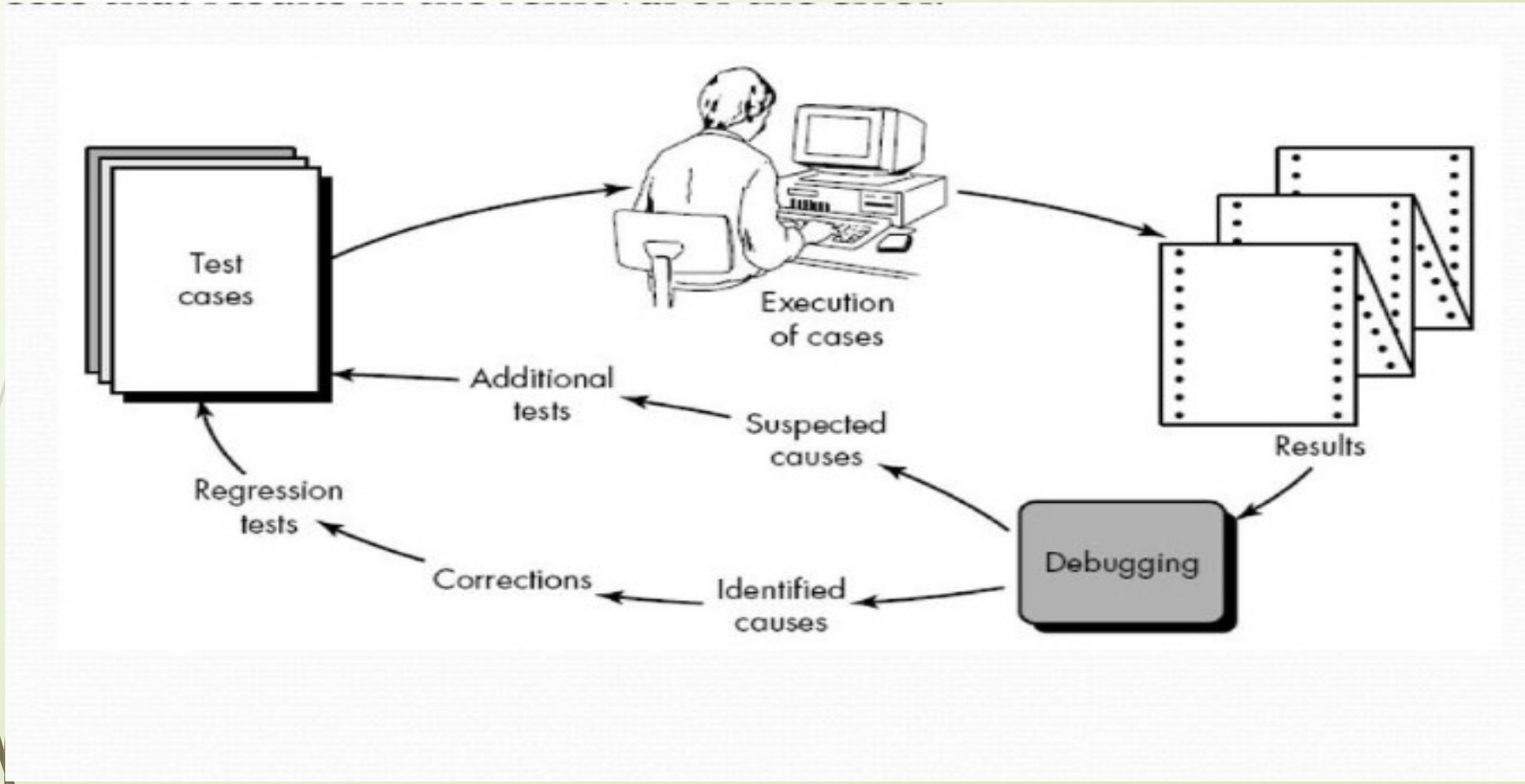
- Logical errors are slipups in implementing the logic to achieve the desired output.
- These errors must be traced and resolved with various data for each functionality.
- Several sophisticated high-end debuggers help trace each variable or data item and support setting various types of break points.
- Logic errors can be difficult to trace and track down.
- A logical error is when the program looks like it is working normally without any errors but it just do the wrong thing.
- Technically the program is correct, but the results won't be what is expected.
- **Debugging techniques** can simply be use to find and resolve logical errors.

Runtime Error

- Runtime errors surface when a program runs into an unexpected condition or situation such as dividing by zero, memory overflow, or addressing a wrong or unauthorized memory location or device, or when a program tries to perform an illegitimate or unauthorized operation or tries to access a library, for example.

- The programs must be thoroughly tested for various types of inputs (valid data sets, invalid data sets and boundary value data sets) and conditions to identify these errors. Once identified, runtime errors are easy to fix.

Debugging Process



Debugging Process

- The debugging process begins with the execution of a test case. Results are assessed and if lack of correspondence between expected and actual performance is encountered then debugging activity will be performed.

- The debugging process will always have one of two outcomes:
  1. The cause will be found and corrected, or
  2. The cause will not be found.

- In the latter case, the person performing debugging may suspect a cause, design a test case to help validate that suspicion, and work toward error correction in an iterative fashion

How Do We Find Bugs?

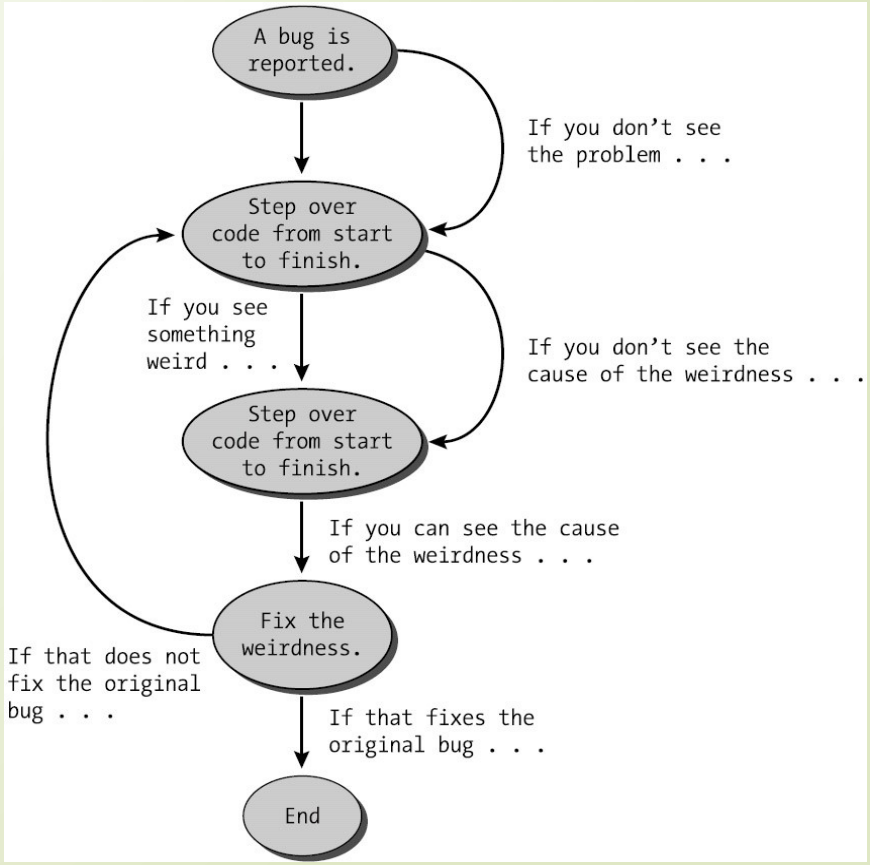
Two main approaches:

1. Static Debugging
   - Brute force
   - Backtracking
   - Cause elimination
2. Dynamic Debugging
   - Using a debugger

Static Debugging

- Brute force: It is the most common debugging technique which is least efficient.

It frequently leads to wasted effort and time.



Static Debugging

- Backtracking: Can be successful in small programs.

This technique begins from the point where the problem (symptoms) has been uncovered, the code is traced backward until the site of the cause is found.

- It is drawback is when the source code line increases, the number of path may become unmanageable.

Static Debugging

- Cause elimination: This introduce the concept of binary partition.
- Uses binary partition to reduce the number of location potential where errors can exist.
- It isolates potential cause, devise cause hypothesis test to isolate bug.

Limitation of Static Debugging

- The problem with static debugging techniques is that there are things you almost cannot do
  - e.g. code in libraries that we didn't write it. But we all use it.
- For this limitations, there is need to embark on, **Dynamic debugging.**

Debug this Program:

```
def divide(a, b):
    return a / b

print(divide(10, 0))
```

Consider this Program and Debug it

```
def calculate_total(price, quantity):
    total = price * quantity
    return total

print(calculate_total(100, -2))
```

Debugging Scenario

Scenario:
A login system allows users with empty passwords.

❓ **Questions:**
1.What type of error is this?
2.How would you debug this issue?
3.Suggest a fix

Software Testing Tools

- **Software Testing Tools:** Are tools or applications that are used for testing a software product in order to assure firmness, thoroughness, and performance of software products.

- These tools are commercial software testing tools. The quality of the software is evaluated by software testers with the help of these testing tools.

Testing Tools

- Testing involves many labor-intensive tasks since it involves running numerous program executions and handling a considerable amount of information.-
- Appropriate tools can alleviate the burden of tedious clerical operations and make them less error prone. Sophisticated tools can support test design and generation, making them more effective.
- Guidance for managers and testers on selecting testing tools is crucial, as the right tool significantly impacts testing efficiency and effectiveness.
- Tool selection depends on diverse factors, such as development choices, evaluation objectives and execution facilities.
- In general, there might not be a unique tool to satisfy specific needs, so a suite of selected tools could be appropriate.

Software Testing Tools

- **Cross-browser Testing Tools:** Cross-browser testing tools helps to perform cross-browser testing that lets the tester check whether the website works as intended when accessed through different browser-OS combinations. For example, Testsigma, Testim, Perfecto, etc.
- **Mobile Testing Tools:** Mobile testing tools are used to test the application for compatibility on different mobile devices. For example, Appium, Robotium, Test IO, etc.

Software Testing Tools

- **Security Testing Tools:** Security testing is used to detect the vulnerabilities and safeguard the application against the malicious attacks. For example, NetSparker, Vega, ImmuniWeb, etc.
- **GUI Testing Tools:** GUI testing tools are used to test the graphical user interface of the software. For example, EggPlant, Squish, AutoIT, etc.
- **Performance Testing Tools:** Performance testing tools helps to perform effectively and efficiently performance testing which is a type of non-functional testing that checks the application for attributes like stability, scalability, performance, speed, etc. For example, WebLOAD, Apache JMeter, Neo Load, etc.

Thank you